

PHB 228: Homework 2

Part 1: Linear and Generalized Linear Models (34 points)

1.1 Linear Models and Diagnostics (16 points)

Using the built-in `mtcars` dataset:

- a) (4 points) Create a linear model predicting `mpg` from `wt` and `hp`. Display the summary and interpret the coefficients.

$\beta_1 \approx 37.23$ is the intercept, which represents the predicted mpg when $wt = 0$ and $hp = 0$, but this isn't meaningful because there are no cars that weighs 0 lbs or has 0 horsepower.

$\beta_2 \approx -3.88$ ($p = 1.12e-06$) is the `wt` coefficient. It holds `hp` constant and each additional 1,000 lbs of weight is associated with about 3.88 mpg fewer mpg. This effect is highly significant.

$\beta_3 \approx -0.032$ ($p = 0.00145$) is the `hp` coefficient. It holds weight constant and each additional unit of horsepower is associated with about 0.03 fewer mpg. This effect is also significant.

```
#?mtcars
m1 <- lm(mpg ~ wt + hp, data = mtcars)
summary(m1)
```

Call:

```
lm(formula = mpg ~ wt + hp, data = mtcars)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.941	-1.600	-0.182	1.050	5.854

Coefficients:

Estimate	Std. Error	t value	Pr(> t)
----------	------------	---------	----------

```

(Intercept) 37.22727    1.59879   23.285   < 2e-16 ***
wt          -3.87783    0.63273   -6.129   1.12e-06 ***
hp          -0.03177    0.00903   -3.519   0.00145 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

Residual standard error: 2.593 on 29 degrees of freedom
Multiple R-squared:  0.8268,    Adjusted R-squared:  0.8148
F-statistic: 69.21 on 2 and 29 DF,  p-value: 9.109e-12

```

- b) (5 points) Create and interpret two diagnostic plots: residuals vs. fitted values, and a normal Q-Q plot.

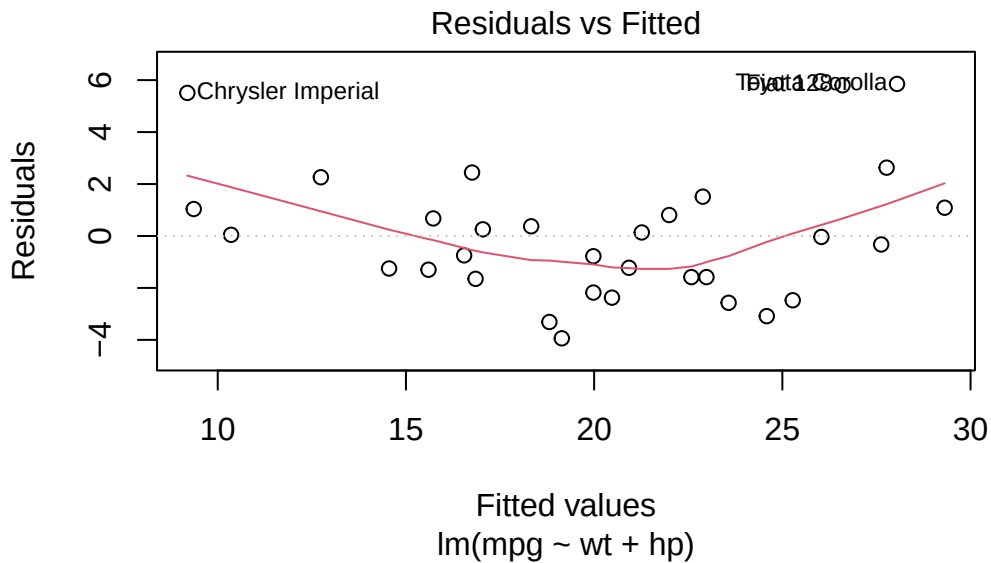
Residuals vs. Fitted:

The red line shows a U or funnel shape rather than a flat horizontal line, which suggests that the linear model is missing a non linear pattern in the data. At low and high fitted values, residuals are mostly positive and mostly negative in the middle, which could mean the relationship between the predictors and mpg may not be strictly linear. Vertical spread is pretty constant across the fitted values, so constant variance assumptions looks acceptable. There are three observations (Chrysler, Fiat, and Toyota) that stand out as having large positive residuals.

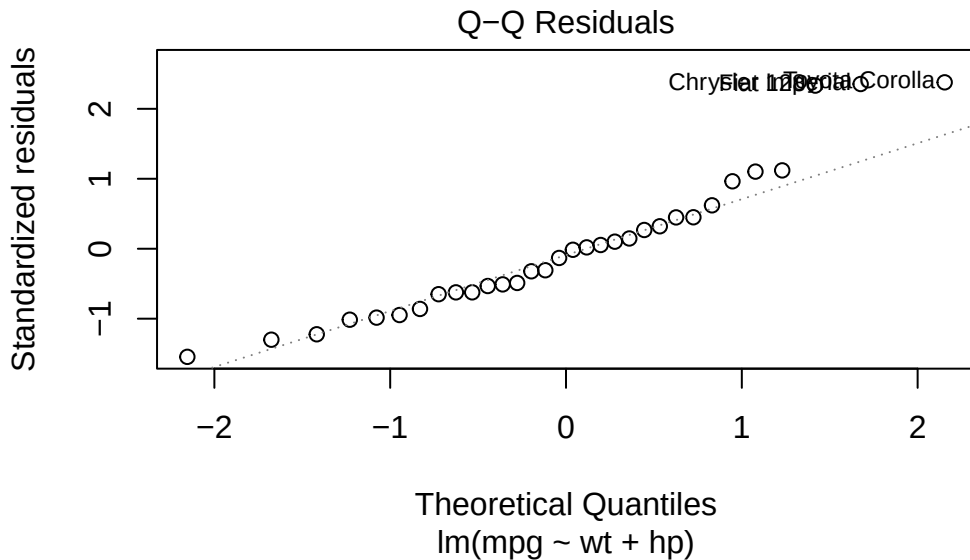
Normal Q-Q

Most residuals fall close to the reference line, which supports the normality assumption. The upper tail deviates a bit, with Chrysler, Fiat, and Toyota shows larger than expected positive residuals. Overall, the residuals are approximately normal, but more analysis needs to be done with the three observations as potential outliers.

```
plot(m1, which = 1) # residual vs. fitted
```



```
plot(m1, which = 2) # normal Q-Q plot
```



c) (3 points) Use `car::vif()` to check for multicollinearity. Interpret the results.

Both predictors (wt and hp) have VIF of approximately 1.77, which is below the common threshold of 5. This means that weight and horsepower are somewhat correlated, mild multicollinearity and does not meaningfully effect the interpretability of the coefficient estimates.

```
library(car)
```

Loading required package: carData

```
vif(m1)
```

```
      wt      hp  
1.766625 1.766625
```

- d) (4 points) Add an interaction term between **wt** and **hp**. Compare to the original model using ANOVA. Which model do you prefer and why?

The ANOVA results gave $F(1,28) = 14.088$, $p = 0.0008108$, which means that the interaction model fits the data significantly better than the additive model. We can also see that the RSS dropped from 195.05 to 129.76 with the addition of the interaction term. I prefer the interaction model (M2) because the improvement in fit is significant and meaningful.

```
m2 <- lm(mpg ~ wt * hp, data = mtcars)  
  
anova(m1, m2)
```

Analysis of Variance Table

Model 1: mpg ~ wt + hp

Model 2: mpg ~ wt * hp

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	29	195.05				
2	28	129.76	1	65.286	14.088	0.0008108 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

1.2 Logistic Regression (18 points)

Using the biopsy dataset from the MASS package:

```
data(biopsy, package = "MASS")  
biopsy <- na.omit(biopsy)  
biopsy$class <- factor(biopsy$class,  
  levels = c("benign", "malignant"))
```

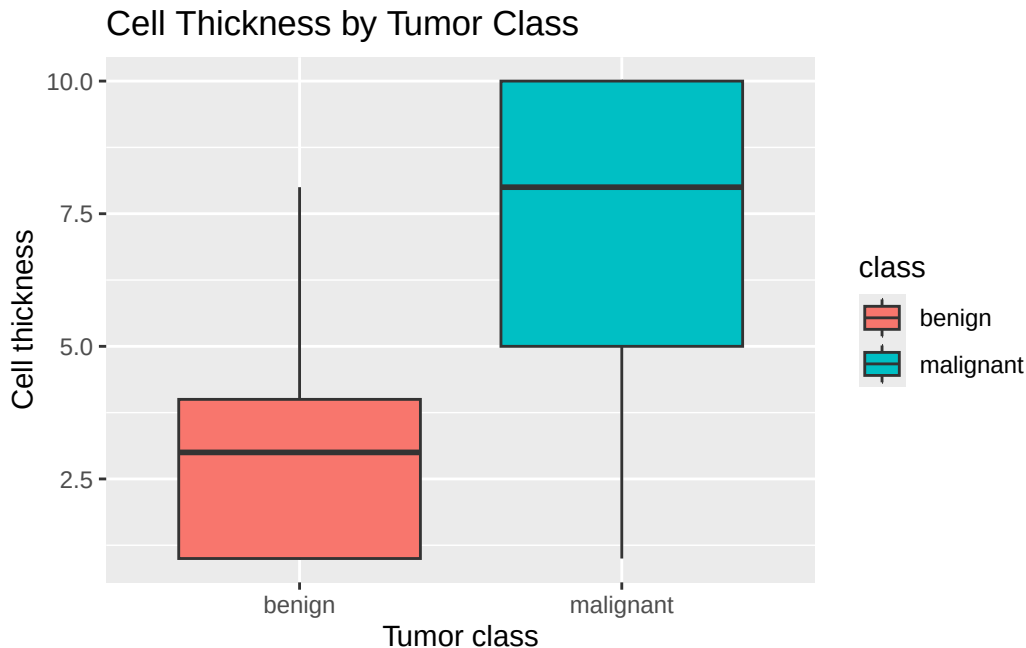
- a) (4 points) Create a boxplot comparing V1 (cell thickness) between benign and malignant tumors. Interpret the plot.

For the benign tumors, the median sits around 3 and a few benign tumors had higher thickness, shown by the whisker reaching up to about 8. The middle 50% of the data (IQR) runs from is about 1 to 4. For malignant tumors, the median is higher at around

8, with the box from about 5 to 10, so most malignant tumors fall in the higher range. There is little overlap between the two groups as the benign box maxes out around 4, while the malignant box starts around 5, which means V1 separates the two groups well. This can mean that cell thickness (V1) can be a strong predictor of malignancy.

```
library(ggplot2)

ggplot(data = biopsy, mapping = aes(x = class, y = V1, fill = class)) +
  geom_boxplot() +
  labs(title = "Cell Thickness by Tumor Class", x = "Tumor class", y = "Cell thickness")
```



- b) (4 points) Fit a logistic regression predicting malignancy from V1, V2, and V3. Display the summary and calculate odds ratios.

All three predictors (V1, V2, V3) are significant predictors of malignancy (all $p < 0.001$). The odds ratios are 1.81 for V1, 1.89 for V2, and 2.06 for V3. Holding the other predictors constant, each one unit increase in V1 multiplies the odds of malignancy by about 1.81 or an 81% increase. For V2, a one unit increase raises the odds by about 89%, and a one unit increase in V3 more than doubles the odds (106% increase). V3 has the strongest effect of the three.

```
m2 <- glm(class ~ V1 + V2 + V3,
          family = binomial(link = "logit"),
          data = biopsy)

summary(m2)
```

```
Call:
glm(formula = class ~ V1 + V2 + V3, family = binomial(link = "logit"),
    data = biopsy)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-7.7210	0.6969	-11.079	< 2e-16 ***
V1	0.5918	0.1030	5.746	9.14e-09 ***
V2	0.6390	0.1704	3.751	0.000176 ***
V3	0.7240	0.1661	4.358	1.31e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 884.35 on 682 degrees of freedom
 Residual deviance: 176.50 on 679 degrees of freedom
 AIC: 184.5

Number of Fisher Scoring iterations: 7

```
exp(coef(m2))
```

	V1	V2	V3
(Intercept)	0.0004433959	1.8073247714	1.8946201232

- c) (6 points) Calculate predicted probabilities, create a confusion matrix (threshold = 0.5), and report accuracy, sensitivity, and specificity.

Using the threshold of 0.5, the confusion matrix gave 430 true negatives (benign correctly identified), 219 true positives (malignant correctly identified), 14 false positives (benign flagged as malignant), and 20 false negatives (malignant missed as benign). Accuracy = 0.950, which means the model correctly classified about 95% of all tumors. Sensitivity = 0.916, which means it correctly identified about 91.6% of the tumors that were actually malignant. Specificity = 0.968, which means it correctly identified about 96.8% of the tumors that were actually benign.

```
pred_prob <- predict(m2, type = "response")

pred_class <- ifelse(pred_prob > 0.5, "malignant", "benign")
pred_class <- factor(pred_class, levels = c("benign", "malignant"))
```

```
cm <- table(Predicted = pred_class, Actual = biopsy$class)
cm
```

	Actual	
Predicted	benign	malignant
benign	430	20
malignant	14	219

```
TN <- cm["benign", "benign"]
FN <- cm["benign", "malignant"]
FP <- cm["malignant", "benign"]
TP <- cm["malignant", "malignant"]
```

```
accuracy <- (TP + TN) / sum(cm)
sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)
```

```
c(accuracy = accuracy, sensitivity = sensitivity, specificity = specificity)
```

accuracy	sensitivity	specificity
0.9502196	0.9163180	0.9684685

- d) (4 points) Predict the probability of malignancy for a new observation with $V1 = 5$, $V2 = 4$, $V3 = 4$. Interpret the result.

The predicted probability of malignancy for a tumor ($V1 = 5$, $V2 = 4$, $V3 = 4$) is about 0.666 or 66.6%. This probability exceeds the 0.5 classification threshold, so this tumor would be classified as malignant. This prediction is only a bit above the threshold, indicating the prediction is not highly confident. I would say this makes sense because the input value is (5,4,4) are a bit elevated, but not as extreme on the one to 10 scale.

```
new_observed <- data.frame(V1 = 5, V2 = 4, V3 = 4)

predict(m2, newdata = new_observed, type = "response")
```

1
0.6660541

Part 2: Mixed-Effects Models (22 points)

You are analyzing reaction time data from a psychology experiment where 20 participants each completed 15 items across two conditions (control and treatment).

- a) (4 points) Create a simulated dataset with the structure described above. Use `set.seed(123)` for reproducibility.

We simulated reaction time data for 20 participants with each completing 15 items across the two conditions, are control and treatment giving us 600 observations in total. To replicate the participant, item, and condition values across the 600 observations, we used `rep()` to get every combination. We built reaction times as the sum of a baseline of 500 ms (could use other values), as a fixed effect of condition (+50 ms for treatment), random participant effects (SD = 50 ms), random participant slopes for condition (SD = 25 ms), random item effects (SD = 30 ms), and residual noise (SD = 40 ms).

```
library(lme4)
```

Loading required package: Matrix

Registered S3 method overwritten by 'lme4':

```
method      from  
na.action.merMod car
```

```
set.seed(123)  
n_participants <- 20  
n_items <- 15  
n_conditions <- 2
```

```
participant <- rep(1:n_participants, each = n_items * n_conditions)  
item <- rep(rep(1:n_items, each = n_conditions), times = n_participants)  
condition <- rep(c("control", "treatment"), times = n_participants * n_items)
```

```
simulation_data <- data.frame(  
  participant = factor(participant),  
  item = factor(item),  
  condition = factor(condition)  
)
```

```
participant_effects <- rnorm(n_participants, mean = 0, sd = 50)  
participant_slopes <- rnorm(n_participants, mean = 0, sd = 25)  
item_effects <- rnorm(n_items, mean = 0, sd = 30)
```

```
simulation_data$rt <- 500 +  
  ifelse(simulation_data$condition == "treatment", 50, 0) +
```



```

participant_effects[simulation_data$participant] +
item_effects[simulation_data$item] +
rnorm(nrow(simulation_data), mean = 0, sd = 40)

#head(simulation_data)
#str(simulation_data)

```

b) (5 points) Fit three models and compare them:

1. Fixed effects only (using `lm()`)
2. Random intercepts for participants only
3. Random intercepts for both participants and items

We compared AIC across the three models, where we got $m1 = 6730.88$, $m2 = 6396.00$, $m3 = 6233.79$. For BIC, $m1 = 6744.07$, $m2 = 6413.59$, $m3 = 6255.77$. The LRT test confirmed that each added random effect significantly improve the fit. $m2_ri$ vs. $m1_fixed_effect$ gave $\chi^2(1) = 325.79$, $p < 0.001$, and $m3_ri$ vs. $m2_ri$ gave $\chi^2(1) = 164.22$, $p < 0.001$. I would say model three is preferred because it has the lowest AIC, BIC, and it accounts for both participant level in item level variability.

```

library(lme4)
m1_fixed_effect <- lm(rt ~ condition, data = simulation_data)
m2_ri <- lmer(rt ~ condition + (1 | participant), data = simulation_data)
m3_ri <- lmer(rt ~ condition + (1 | participant) + (1 | item), data = simulation_data)

AIC(m1_fixed_effect, m2_ri, m3_ri)

```

	df	AIC
m1_fixed_effect	3	6730.879
m2_ri	4	6396.004
m3_ri	5	6233.786

```
BIC(m1_fixed_effect, m2_ri, m3_ri)
```

	df	BIC
m1_fixed_effect	3	6744.070
m2_ri	4	6413.592
m3_ri	5	6255.770

```
anova(m2_ri, m3_ri)
```

refitting model(s) with ML (instead of REML)

```
Data: simulation_data
Models:
m2_ri: rt ~ condition + (1 | participant)
m3_ri: rt ~ condition + (1 | participant) + (1 | item)
      npar    AIC    BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
m2_ri    4 6407.1 6424.7 -3199.5    6399.1
m3_ri    5 6244.9 6266.9 -3117.4    6234.9 164.22  1 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
anova(m2_ri, m1_fixed_effect)
```

```
refitting model(s) with ML (instead of REML)
```

```
Data: simulation_data
Models:
m1_fixed_effect: rt ~ condition
m2_ri: rt ~ condition + (1 | participant)
      npar    AIC    BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
m1_fixed_effect    3 6730.9 6744.1 -3362.4    6724.9
m2_ri              4 6407.1 6424.7 -3199.5    6399.1 325.79  1 < 2.2e-
16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- c) (5 points) Extract and interpret the fixed effects from your best model. Calculate the intraclass correlation coefficient (ICC) for participants.

It gave a fixed effect intercept of 506.40 ms and a treatment effect of 56.27 ms. The intercept represents the predicted reaction time in the control condition for an average participant on an average item. The treatment condition adds about 56.3 ms to reaction time compared to control, while holding the other factors constant. The t values 39.80 and 17.36 mean they are highly significant, but it is important to know that lme4 does not report p values. For the variant component we get participant variance = 2209.63, item variance = 692.86, and residual variance = 1576.17. The ICC value is 0.493, which means about 49.3% of the total variation in reaction times comes from the differences between participants.

```
summary(m3_ri)$coefficients
```

	Estimate	Std. Error	t value
(Intercept)	506.39863	12.725011	39.79553
conditiontreatment	56.27373	3.241578	17.35998

```
var_comp <- as.data.frame(VarCorr(m3_ri))
var_comp
```

```
      grp      var1 var2      vcov      sdcor
1 participant (Intercept) <NA> 2209.6256 47.00665
2      item (Intercept) <NA>  692.8608 26.32225
3   Residual      <NA> <NA> 1576.1743 39.70106
```

```
participant_var <- var_comp$vcov[var_comp$grp == "participant"]
item_var <- var_comp$vcov[var_comp$grp == "item"]
residual_var <- var_comp$vcov[var_comp$grp == "Residual"]

icc_participant <- participant_var / (participant_var + item_var + residual_var)
icc_participant
```

```
[1] 0.4933675
```

- d) (5 points) Fit a model with random slopes for condition by participant. Compare to the random intercepts model using a likelihood ratio test.

For LRT we compared and four to M3, we used RMEL since the models differ only in random affects. The results gave $\chi^2(2) = 2.39$ and $p = 0.302$. AIC increase the bit from 6233.8 to 623.4 and the BIC increased from 6255.8 to 6266.2, which means there was no improvement in the fit. This means we prefer the random intercepts model m3. Even though there are some variation in the treatment effects, it is not large enough to support the extra parameters that the random slopes model requires.

```
m4 <- lmer(rt ~ condition + (1 + condition | participant) + (1 | item),
           data = simulation_data)
```

boundary (singular) fit: see help('isSingular')

```
anova(m3_ri, m4, refit = FALSE)
```

Data: simulation_data

Models:

m3_ri: rt ~ condition + (1 | participant) + (1 | item)

m4: rt ~ condition + (1 + condition | participant) + (1 | item)

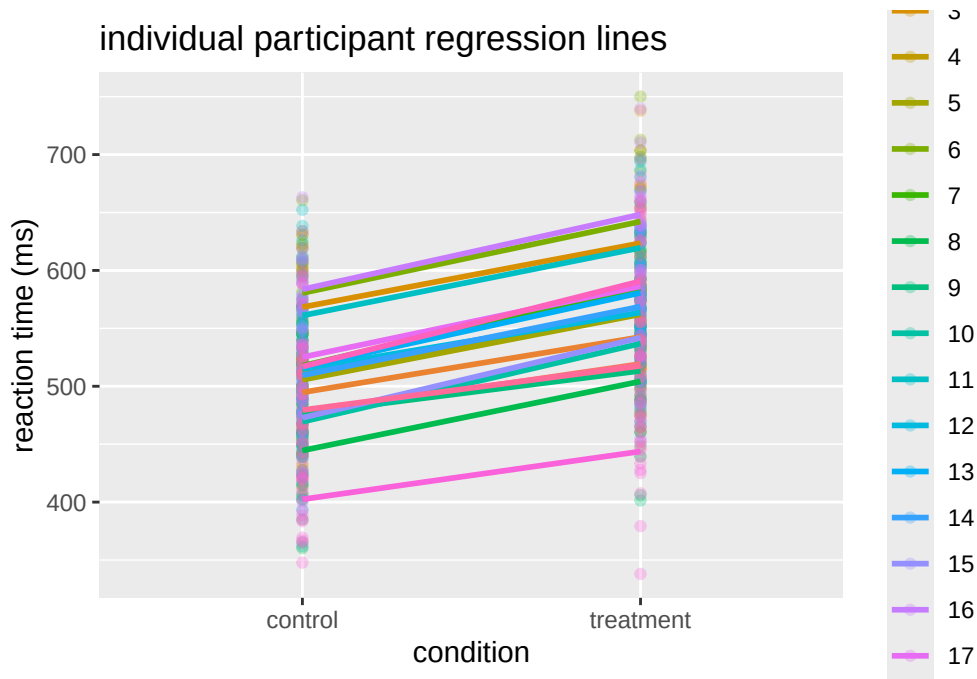
	npar	AIC	BIC	logLik	-2*log(L)	Chisq	Df	Pr(>Chisq)
m3_ri	5	6233.8	6255.8	-3111.9	6223.8			
m4	7	6235.4	6266.2	-3110.7	6221.4	2.3947	2	0.302

- e) (3 points) Create a plot showing individual participant regression lines.

The plot shows the individual regression lines for each of the 20 participants from control to treatment. Most of the lines go upward, which matches the overall treatment effect of about +56 ms. The regression lines vary in both their intercepts (baseline reaction times range from 440 ms to 590 ms) and their slopes. Some of the participants show treatment effects of over 100 ms while others show effects of less than 50 ms. This shows some variability in the participant level random effects, where different participants have different baseline and different sensitivities to the treatment.

```
ggplot(data = simulation_data, data, mapping = aes (x = condition, y = rt, group = participant)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "individual participant regression lines", x = "condition", y = "reaction time (ms)")
```

`geom\_smooth()` using formula = 'y ~ x'



Part 3: Advanced Data Visualization (22 points)

Using the palmerpenguins dataset:

3.1 Custom Theme (4 points)

Create a custom theme called `theme_penguin` based on `theme_minimal()` or `theme_classic()`. It should have a professional appearance suitable for scientific publication and include at least 5 customized theme elements. Use this theme for all subsequent plots.

```
library(palmerpenguins)
```

Attaching package: 'palmerpenguins'

The following objects are masked from 'package:datasets':

```
penguins, penguins_raw
```

```
library(dplyr)
```

Attaching package: 'dplyr'

The following object is masked from 'package:car':

```
recode
```

The following objects are masked from 'package:stats':

```
filter, lag
```

The following objects are masked from 'package:base':

```
intersect, setdiff, setequal, union
```

```
df <- penguins
#head(df)

df <- na.omit(df)
#head(df)

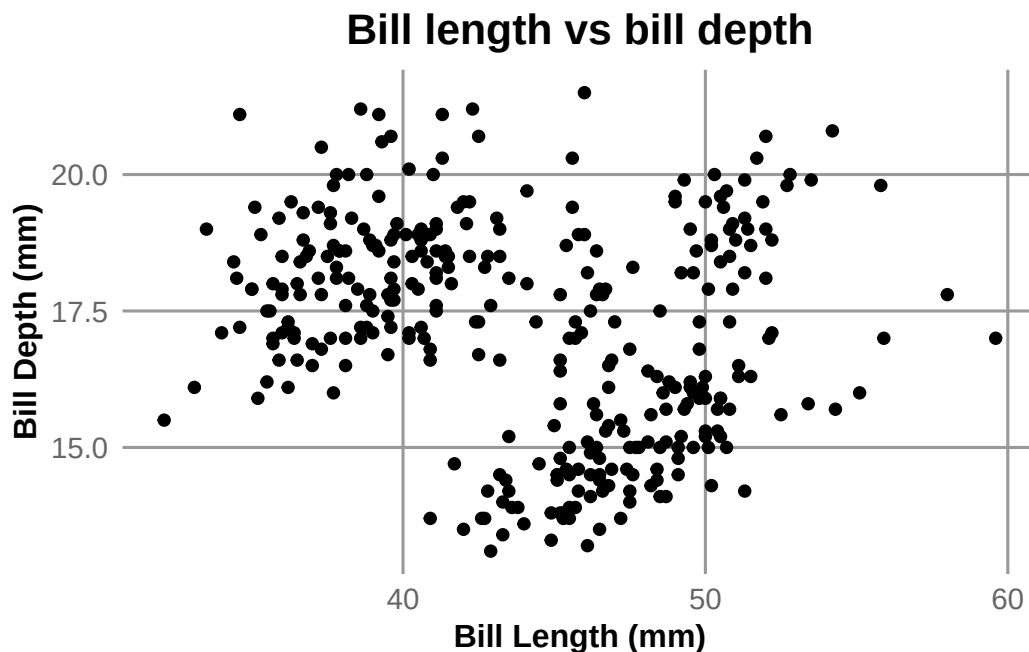
theme_penguin <- function(base_size = 12) {
```

```

theme_minimal(base_size = base_size) +
  theme(
    plot.title = element_text(size = rel(1.3), face = "bold", hjust = 0.5),
    plot.subtitle = element_text(size = rel(1.0), hjust = 0.5, color = "grey20"),
    axis.title = element_text(size = rel(1.0), face = "bold"),
    axis.text = element_text(size = rel(0.9), color = "gray40"),
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(color = "gray60"),
    legend.position = "bottom"
  )
}

ggplot(data = df, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point() +
  labs(title = "Bill length vs bill depth", x = "Bill Length (mm)", y = "Bill Depth (mm)") +
  theme_penguin()

```



3.2 Faceted Plot (6 points)

Create a faceted scatter plot of bill length vs. flipper length:

- Facet by species using `facet_wrap()`
- Add a linear regression line with 95% CI for each facet

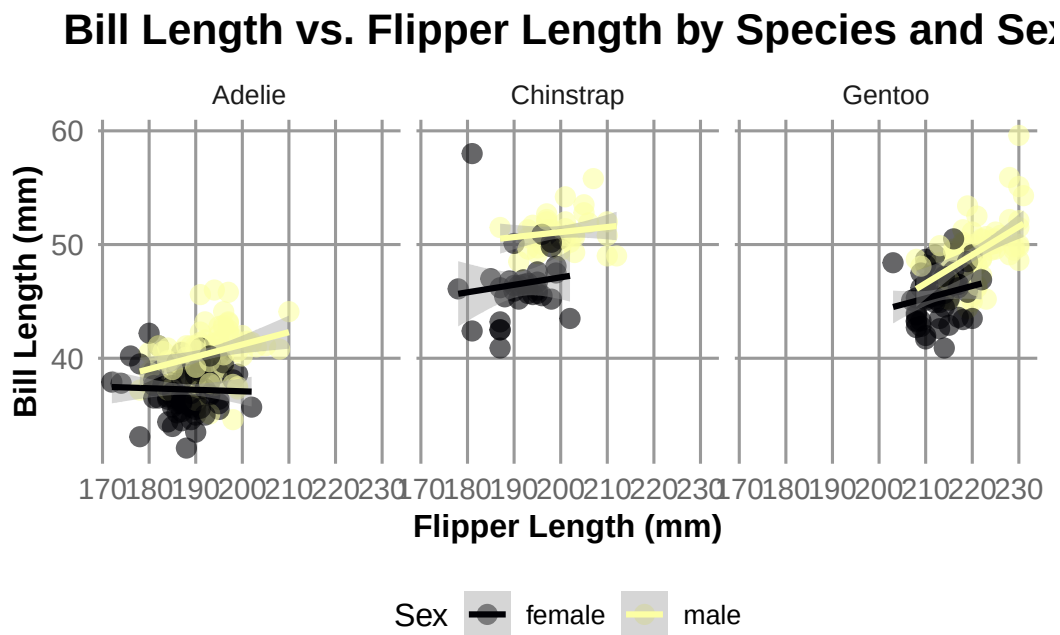
- Color points by sex using a colorblind-friendly palette
- Apply your custom theme

```
library(viridis)
```

Loading required package: viridisLite

```
ggplot(data = df, mapping = aes(x = flipper_length_mm, y = bill_length_mm, color = sex)) +
  geom_point(alpha = 0.6, size = 3) +
  geom_smooth(method = "lm", se = TRUE, level = 0.95) +
  facet_wrap(~ species) +
  scale_color_viridis_d(option = "B") +
  labs(title = "Bill Length vs. Flipper Length by Species and Sex",
       x = "Flipper Length (mm)",
       y = "Bill Length (mm)",
       color = "Sex") +
  theme_penguin()
```

`geom\_smooth()` using formula = 'y ~ x'

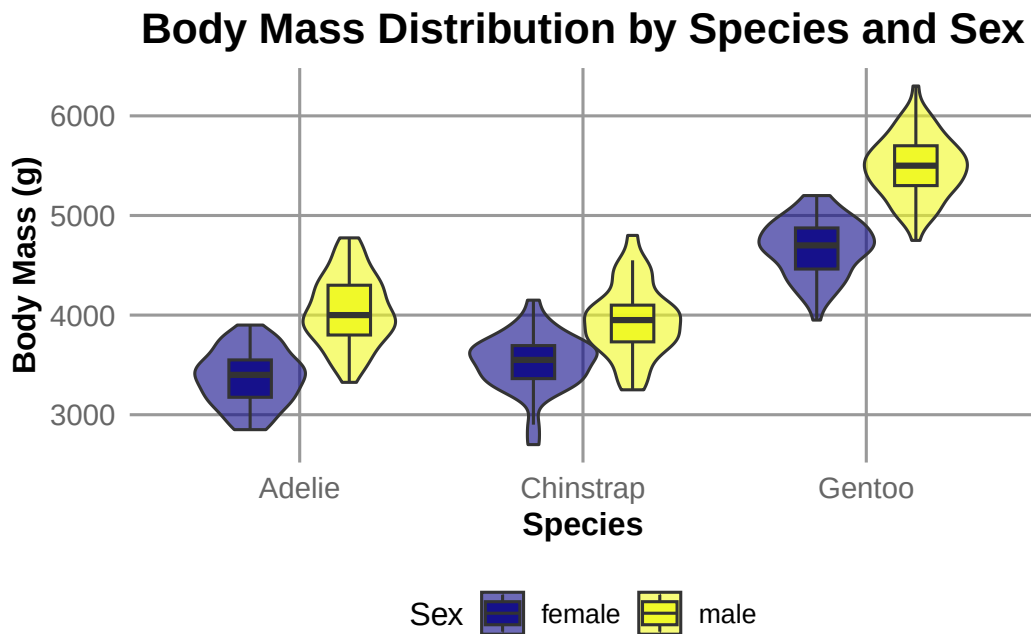


3.3 Distribution Comparison (5 points)

Create a visualization of body mass by species and sex:

- Choose an appropriate geom (boxplot, violin, or both)
- Customize the y-axis to start at 0
- Apply your custom theme

```
ggplot(data = df, mapping = aes(x = species, y = body_mass_g, fill = sex)) +  
  geom_violin(alpha = 0.6, position = position_dodge(width = 0.7)) +  
  geom_boxplot(width = 0.3, position = position_dodge(width = 0.7),  
              outlier.shape = NA, alpha = 0.9) +  
  # expand removes extra padding below 0  
  scale_fill_viridis_d(option = "C") +  
  labs(title = "Body Mass Distribution by Species and Sex",  
       x = "Species",  
       y = "Body Mass (g)",  
       fill = "Sex") +  
  theme_penguin()
```



3.4 Complex Multi-Variable Visualization (7 points)

Create a comprehensive visualization that:

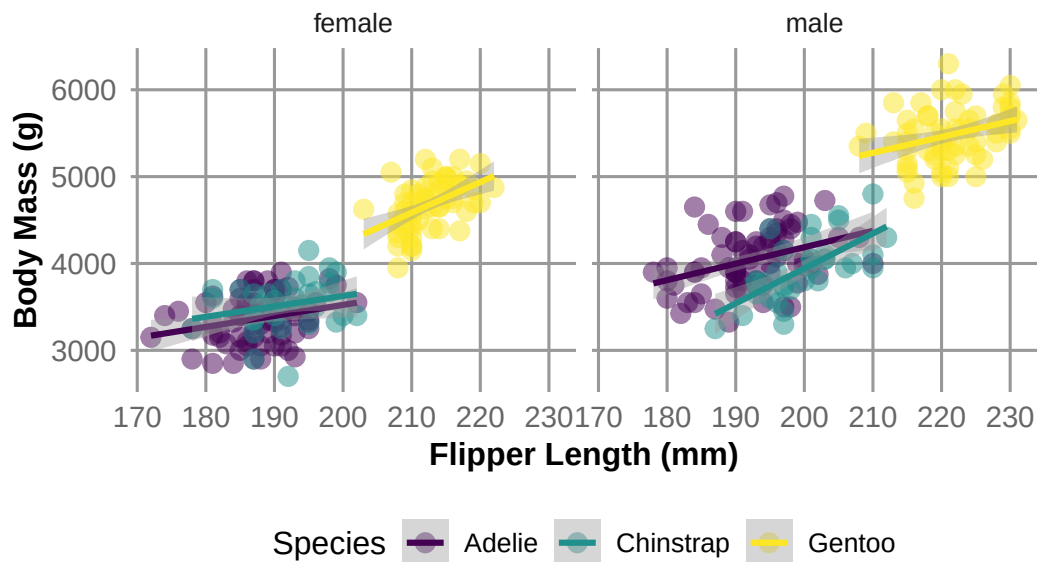
- Uses a combination of at least two geoms
- Includes at least three variables
- Applies your custom theme with appropriate scales and legends
- Include a brief paragraph (3-5 sentences) interpreting the biological patterns shown

Below shows the body mass plotted against flipper length, where one panel for each sex and points, and the regression lines are colored by species. For all the three species and both sex, we can see a clear positive relationship between flipper length and body mass, where penguins with longer flippers tend to be heavier. We can also see that Gentoo penguins are the largest species that occupied the upper right region of both panels with its longest flippers around 205 to 230 mm and highest body mass (4500 to 6500 g. We can also see that Adelie and Chinstrap penguins heavily overlap in both its flipper length and body mass. This means that they are more similar in size to each other than is to Gentoo. it is also shown that the pattern is shifted more upwards in the male panel compared to the female panel, meaning that males are heavier than females across the three species.

```
ggplot(data = df, mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point(alpha = 0.5, size = 3) +
  geom_smooth(method = "lm", se = TRUE) +
  facet_wrap(~ sex) +
  scale_color_viridis_d(option = "D") +
  labs(title = "Body Mass vs. Flipper Length by Species and Sex",
       x = "Flipper Length (mm)",
       y = "Body Mass (g)",
       color = "Species") +
  theme_penguin()
```

`geom\_smooth()` using formula = 'y ~ x'

Body Mass vs. Flipper Length by Species and Sex



Part 4: Interactive Visualization with Shiny (12 points)

Build a simple Shiny application that explores the `mtcars` dataset.

a) (5 points) Create a Shiny app with the following UI elements:

- A `selectInput()` dropdown that lets the user choose a predictor variable from `wt`, `hp`, `disp`, and `drat`
- A `plotOutput()` to display a scatter plot
- A `verbatimTextOutput()` to display a model summary

```
library(shiny)

ui <- fluidPage(
  titlePanel("linear regression Explorer for mtcars dataset"),
  sidebarLayout(
    sidebarPanel(
      selectInput(
        inputId = "predictor",
        label = "Choose predictor variable:",
        choices = c("Weight (wt)" = "wt",
                    "Horsepower (hp)" = "hp",
```

```

        "Displacement (disp)" = "disp",
        "Rear axle ratio (drat)" = "drat"),
      selected = "wt"
    )
  ),
  mainPanel(
    plotOutput("scatter_plot"),
    verbatimTextOutput("model_summary")
  )
)
)

```

b) (3 points) In the server function:

- Use a `reactive()` expression to fit a linear model predicting mpg from the user-selected predictor
- Use `renderPlot()` to display a scatter plot of the selected predictor vs. mpg with a regression line
- Use `renderPrint()` to display the `summary()` of the fitted model

```

server <- function(input, output){
  # reactive: fit a linear model to predict mpg from selected predictor
  model <- reactive({
    formula <- as.formula(paste("mpg ~", input$predictor))
    lm(formula, data = mtcars)
  })

  # create scatter plot of the selected predictor with regression line
  output$scatter_plot <- renderPlot({
    plot(mtcars[[input$predictor]], mtcars$mpg,
         xlab = input$predictor,
         ylab = "mpg",
         main = paste("mpg vs.", input$predictor),
         pch = 15, col = "blue")
    abline(model(), col = "red", lws = 3)
  })

  # model the summary
  output$model_summary <- renderPrint({
    summary(model())
  })
}

```

```
shinyApp(ui = ui, server = server)
```

- c) (4 points) Add one enhancement of your choice (e.g., a `sliderInput()` to filter by number of cylinders, a second output showing residual diagnostics, or a `checkboxInput()` to toggle the regression line). Briefly explain your design choice in a comment (2-3 sentences).

I added a second plot output that shows the residuals vs. fitted values and a normal Q-Q plot. I really wanted to add these residual diagnostics because it is important to always look over if the assumptions hold when fitted a model, so in this case whether linearity and normality assumptions hold.

```
# update ui, added residual diagnostics
ui <- fluidPage(
  titlePanel("linear regression Explorer for mtcars dataset"),
  sidebarLayout(
    sidebarPanel(
      selectInput(
        inputId = "predictor",
        label = "Choose predictor variable:",
        choices = c("Weight (wt)" = "wt",
                     "Horsepower (hp)" = "hp",
                     "Displacement (disp)" = "disp",
                     "Rear axle ratio (drat)" = "drat"),
        selected = "wt"
      )
    ),
    mainPanel(
      plotOutput("scatter_plot"),
      verbatimTextOutput("model_summary"),
      plotOutput("residual_plot")
    )
  )
)

# update server
server <- function(input, output) {
  model <- reactive({
    formula <- as.formula(paste("mpg ~", input$predictor))
    lm(formula, data = mtcars)
  })

  output$scatter_plot <- renderPlot({
```

```

plot(mtcars[[input$predictor]], mtcars$mpg,
     xlab = input$predictor,
     ylab = "mpg",
     main = paste("mpg vs.", input$predictor),
     pch = 19, col = "blue")
abline(model(), col = "red", lwd = 3)
})

output$model_summary <- renderPrint({
  summary(model())
})

output$residual_plot <- renderPlot({
  par(mfrow = c(1, 2))
  # diagnostics plots
  plot(model(), which = 1) # residual vs. fitted
  plot(model(), which = 2) # norm Q-Q
})
}

shinyApp(ui = ui, server = server)

```

Part 5: Parallel Computing (10 points)

5.1 System Setup and Benchmarking (4 points)

- a) (2 points) Determine how many cores your computer has (logical and physical). Create a variable `safe_cores` and explain your choice.

My computer has 8 logical cords and 8 physical cores. The number of safe cores I decided on is 7 cores (`detectCores()` - 1) because following lecture notes 17, it is good to leave one core free so the operating system and other applications can continue to run smoothly while R does parallel computing.

```

library(parallel)
lc <- detectCores(logical = TRUE)
lc

```

```
[1] 8
```

```
pc <- detectCores(logical = FALSE)
pc
```

```
[1] 8
```

```
sc <- detectCores() - 1
sc
```

```
[1] 7
```

- b) (2 points) Create a function `slow_mean()` that takes a numeric vector, adds a small delay with `Sys.sleep(0.01)`, and returns the mean. Use `microbenchmark()` to compare sequential (`lapply`) vs. parallel (`mclapply` or `parLapply`) for calculating means of 100 random vectors. Compare median times.

Used `microbenchmark()` with 5 replications on 100 random vectors. The median execution time for sequential (`lapply()`) was 1210.4 ms and for parallel (`mclapply()` with 7 cores) took 193.39 ms, which is about a 6.9x speedup. This shows that parallel processing really does reduce runtime when the work contains many independent slow tasks.

```
library(microbenchmark)

set.seed(123)

slow_mean <- function(x){
  Sys.sleep(0.01)
  mean(x)
}

# create 100 random vectors to compute means
v1 <- lapply(1:100, function(i) rnorm(1000))

# seq vs. parallel
br <- microbenchmark(
  sequential = lapply(v1, slow_mean),
  parallel = mclapply(v1, slow_mean, mc.cores = sc),
  times = 5
)
```

Warning in `microbenchmark(sequential = lapply(v1, slow_mean), parallel = mclapply(v1, : less accurate nanosecond times to avoid potential integer overflows`

```
print(br)
```

Unit: milliseconds

	expr	min	lq	mean	median	uq	max	neval
	sequential	1201.7897	1219.3806	1231.279	1221.2910	1222.1165	1291.8182	5
	parallel	171.0804	175.4686	178.267	175.6704	182.7702	186.3455	5

```
summary(br)
```

	expr	min	lq	mean	median	uq	max	neval
1	sequential	1201.7897	1219.3806	1231.279	1221.2910	1222.1165	1291.8182	5
2	parallel	171.0804	175.4686	178.267	175.6704	182.7702	186.3455	5

5.2 Parallel Bootstrap (6 points)

- a) (2 points) Create `bootstrap_ci_sequential()` with parameters: `data` (numeric vector), `n_bootstrap` (default 1000), `confidence` (default 0.95). Return a named vector with lower and upper bounds.

```
bootstrap_ci_sequential <- function(data, n_bootstrap = 1000, confidence = 0.95){  
  # bootstrap loop  
  bm <- sapply(1:n_bootstrap, function(i){  
    sample_data <- sample(data, size = length(data), replace = TRUE)  
    mean(sample_data)  
  })  
  
  # calc CI bounds  
  alpha <- 1 - confidence  
  lower <- quantile(bm, probs = alpha/2)  
  upper <- quantile(bm, probs = 1- alpha/2)  
  c(lower = unname(lower), upper = unname(upper))  
}
```

- b) (2 points) Create `bootstrap_ci_parallel()` with the same interface but using parallel processing.

```
bootstrap_ci_parallel <- function(data, n_bootstrap = 1000, confidence = 0.95){  
  # bootstrap loop in parallel  
  bm <- unlist(mclapply(1:n_bootstrap, function(i){  
    sample_data <- sample(data, size = length(data), replace = TRUE)  
    mean(sample_data)  
  })
```

```

}, mc.cores = sc))

alpha <- 1 - confidence
lower <- quantile(bm, probs = alpha/2)
upper <- quantile(bm, probs = 1- alpha/2)
c(lower = unname(lower), upper = unname(upper))
}

```

- c) (2 points) Test both on `mtcars$mpg` with bootstrap sample sizes of 500, 1000, and 2000. Create a summary showing how the time difference scales with problem size. Comment on findings.

The median sequential times were 6.32 ms, 11.95 ms, 24.13 ms, and for parallel it was 36.31 ms, 38.05 ms, 39.83 ms. Across the three sizes, the parallel version was actually slower than the sequential version by about 5.7x at $n = 500$, but only 1.7x at $n = 2000$. This is because for each bootstrap iteration it only has a single `sample()` and `mean()` call, which is really fast in microseconds. To set up for parallel takes time and when each task is really small, the setup time is bigger than the time saved by splitting the work, so it spends more time organizing than it would have spent just doing the work sequentially. This means that parallel processing really only helps when the individual tasks is large enough to outweigh the setup cost.

```

size <- c(500, 1000, 2000)

final_results <- microbenchmark(
  seq_500 = bootstrap_ci_sequential(mtcars$mpg, n_bootstrap = 500),
  par_500 = bootstrap_ci_parallel(mtcars$mpg, n_bootstrap = 500),
  seq_1000 = bootstrap_ci_sequential(mtcars$mpg, n_bootstrap = 1000),
  par_1000 = bootstrap_ci_parallel(mtcars$mpg, n_bootstrap = 1000),
  seq_2000 = bootstrap_ci_sequential(mtcars$mpg, n_bootstrap = 2000),
  par_2000 = bootstrap_ci_parallel(mtcars$mpg, n_bootstrap = 2000),
  times = 5
)

print(final_results)

```

Unit: milliseconds

	expr	min	lq	mean	median	uq	max	neval
seq_500		5.515074	5.756646	5.789618	5.796129	5.906747	5.973495	5
par_500		17.150300	17.701217	19.975897	18.385220	19.017932	27.624816	5
seq_1000		10.856882	11.707550	13.706079	12.673674	16.540712	16.751575	5
par_1000		18.871767	22.535773	26.530690	22.690917	22.723635	45.831358	5
seq_2000		20.296722	20.794995	21.591592	20.973837	21.256696	24.635711	5

par_2000	23.016785	23.413173	25.363125	25.768336	26.960452	27.656878	5
----------	-----------	-----------	-----------	-----------	-----------	-----------	---